

ProjectPilot AI

The Complete Frontend Architecture & Design Guide

Next.js · Tailwind CSS · State Management · Color System
Every page, exactly how it should look and how to build it

The ONLY frontend reference this team needs.
B.E. Computer Engineering — Mumbai University · Team of 4

1. The Stack, and Why Each Piece Was Chosen

Piece	Choice	Why this, specifically, for this project
Framework	Next.js 14 (App Router), TypeScript	File-based routing matches our page list directly; server components reduce the API calls the dashboard needs on load; TypeScript catches schema mismatches against our Supabase types before runtime.
Styling	Tailwind CSS	Utility classes keep 4 people's UI code visually consistent without a shared CSS file everyone has to edit.
Component library	shadcn/ui (Radix + Tailwind)	Not a black-box library — it copies real component code into your repo, so you can freely restyle it with our color palette. Free, accessible, and covers everything we need (Dialog, Tabs, Table, Select, Toast).
Server state	TanStack Query (React Query)	Our data (risk scores, tickets, reports) lives on the server and changes from multiple places (webhooks, background jobs, other users). React Query handles caching, background refetching, and loading/error states for this automatically — writing this by hand for 12 pages would eat weeks.
Client/UI state	Zustand	For things that are NOT server data: sidebar collapsed, active filters, simulation form drafts, chat input text. Zustand has near-zero boilerplate compared to Redux, and unlike Context API, doesn't cause unnecessary re-renders across a data-heavy dashboard.
Charts	Recharts	Covers every chart we need (workload bars, risk trends, simulation histograms) with a simple React API.
Forms	react-hook-form + zod	Type-safe validation for the task-generator input, simulation scenario form, and settings forms, with minimal re-renders.
Icons	lucide-react	Consistent, clean icon set that matches shadcn/ui by default.
Fonts	Inter (UI) + JetBrains Mono (code/technical)	Inter is built for dashboards and renders numbers/percentages clearly, which matters constantly on a risk-score-heavy UI.

Why Zustand + React Query together, not just one? They solve two different problems. React Query answers "what does the server say right now, and keep it fresh." Zustand answers "what is this specific browser tab doing right now." Mixing server data into Zustand (or vice versa) is the most common state-management mistake teams make — keep this split strict from day one.

1.1 The "MVC" Mapping (for a team used to thinking in MVC)

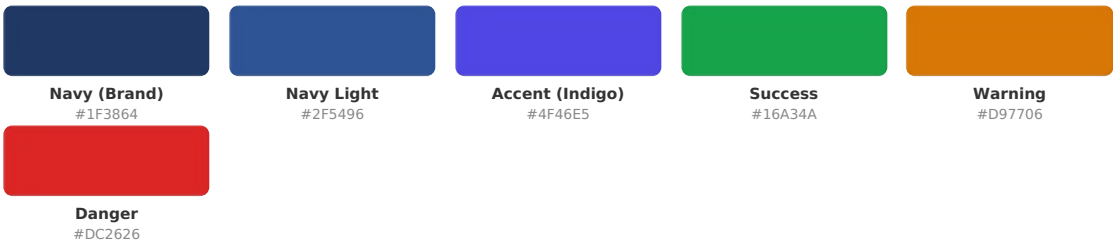
Next.js isn't literally MVC, but the same separation of concerns maps cleanly:

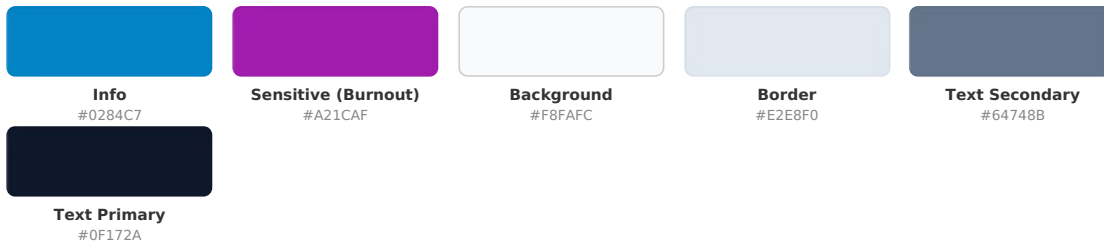
MVC Concept	Where it lives here	What goes there
Model	<code>types/</code> , <code>lib/api/</code> , <code>store/</code>	TypeScript types matching Supabase tables, functions that fetch/write data, Zustand stores holding client state.
View	<code>components/</code> , <code>app/**/page.tsx</code>	Pure UI — components receive data as props and render it. No data-fetching logic inside these files directly.
Controller	<code>hooks/</code> , <code>app/api/</code>	Custom React Query hooks that connect Model to View, and Next.js route handlers/server actions that talk to FastAPI or Supabase.

2. Color Palette & Design System

One rule drove every color choice here: **this is a risk-communication tool first, a dashboard second**. Red/amber/green has to mean the same thing everywhere in the app, with zero exceptions.

2.1 Core Palette





Why these, specifically

- **Navy (#1F3864)** — carried over from your project documentation and abstract, so the brand is consistent across your report, PDF guides, and the actual product. Conveys engineering seriousness, not a toy app.
- **Indigo accent (#4F46E5)**, not navy, for buttons/links — if everything interactive were also navy, nothing would stand out against the sidebar. The accent is reserved *only* for things you can click or that are actively selected.
- **Red/Amber/Green** are used *exclusively* for risk severity, everywhere in the app — never reuse these three for anything decorative. The moment red means something other than "high risk" somewhere in the UI, the whole point of Explainable Risk Scores is undermined.
- **Purple (#A21CAF)** is reserved only for burnout/workload signals — a deliberate visual cue that this data is sensitive and lead-only, distinct from ordinary risk types.

2.2 Typography

Use	Font	Sizes
All UI text	Inter (via next/font/google)	Page titles 20–24px · section titles 14–16px · body 12–13px · labels/meta 10–11px
Code, SHAP JSON, technical report text	JetBrains Mono	11–12px

2.3 Component Style Rules

- **Cards:** white background, 1px `#E2E8F0` border, 8–10px radius, subtle shadow. This is the single container style used everywhere — stat cards, tables, panels.
- **Risk badges:** pill-shaped, solid color fill matching severity (red $\geq 70\%$, amber 40–69%, green $< 40\%$) — the exact same three thresholds everywhere in the app, hardcoded once in `lib/riskColor.ts`, never re-implemented per page.
- **Buttons:** primary = solid indigo; secondary = white with navy border; destructive = solid red. Never use red for anything except a genuinely destructive or high-risk action.
- **Sidebar:** fixed, navy background, active item highlighted with a lighter navy block + indigo left border.

3. Folder Structure

```
projectpilot-frontend/
├─ app/
│   ├─ (auth)/login/page.tsx
│   └─ (dashboard)/
│       ├─ layout.tsx           # sidebar + topbar shell
│       ├─ dashboard/page.tsx
│       ├─ sprints/board/page.tsx
│       ├─ sprints/plan/page.tsx
│       ├─ tickets/[id]/page.tsx
│       ├─ meetings/page.tsx
│       ├─ meetings/[id]/page.tsx
│       ├─ risk/page.tsx
│       ├─ workload/page.tsx
│       ├─ simulation/page.tsx
│       ├─ reports/page.tsx
│       ├─ chat/page.tsx
│       └─ settings/integrations/page.tsx
├─ api/                        # route handlers (proxy to FastAPI if needed)
├─ components/
│   ├─ ui/                     # shadcn/ui primitives (Button, Card, Tabs, Dialog...)
│   └─ features/               # composed, feature-specific components
│       ├─ dashboard/StatCard.tsx, TopRisksList.tsx
│       ├─ tickets/RiskExplainPanel.tsx
│       └─ simulation/ScenarioForm.tsx, ProbabilityChart.tsx
│           └─ risk/RiskTable.tsx
├─ hooks/                     # React Query hooks: useTickets, useRiskScores, useSimulation...
├─ lib/
│   ├─ api/                   # fetch functions per resource (tickets.ts, risk.ts...)
│   ├─ supabase/client.ts, server.ts
│   ├─ riskColor.ts          # the ONE place severity thresholds/colors are defined
│   └─ utils.ts
├─ store/                     # Zustand stores: useUIStore, useFilterStore, useChatDraftStore
├─ types/                     # TypeScript types matching Supabase tables
└─ styles/globals.css
```

3.1 Example: how a page actually pulls this together

```
// hooks/useTickets.ts (Controller)
export function useTickets(sprintId: string) {
  return useQuery({
    queryKey: ['tickets', sprintId],
    queryFn: () => fetchTickets(sprintId), // lib/api/tickets.ts — Model
  })
}

// app/(dashboard)/sprints/board/page.tsx (View)
export default function SprintBoardPage() {
  const { data: tickets, isLoading } = useTickets(currentSprintId)
  return <KanbanBoard tickets={tickets} loading={isLoading} />
}
```

4. State Management — Exact Split

Data	Tool	Reason
Tickets, risk scores, reports, notifications, workload data	React Query	Lives on the server, changes from webhooks/background jobs, needs caching + refetch
Sidebar collapsed/expanded	Zustand	Pure UI state, never touches the server
Active table filters (risk type, sprint)	Zustand	Local to the browser tab, resets on reload is fine
Simulation scenario form (before submit)	Zustand or local <code>useState</code>	Draft input, not saved until "Run Simulation" is clicked
Chat message draft + scroll position	Zustand	UI-only, until a message is actually sent (then it becomes a React Query mutation)
Logged-in user session	Supabase Auth (not Zustand)	Supabase's own client already manages this — don't duplicate it into a store

5. Page-by-Page Specification

Every page below follows the same shell: fixed navy sidebar (220px) with the 11 nav items, a top bar (search + notifications + user), and the main content area on `#F8FAFC` background. Only the main content differs per page.

5.1 Login Page — `/login`

Purpose: single entry point, GitHub OAuth only (no email/password — simpler and matches Section 6 of the database guide).

Layout: centered card on a plain navy-to-white gradient background — logo, one-line pitch, a single "Continue with GitHub" button.

State: none needed beyond Supabase Auth's own session handling.

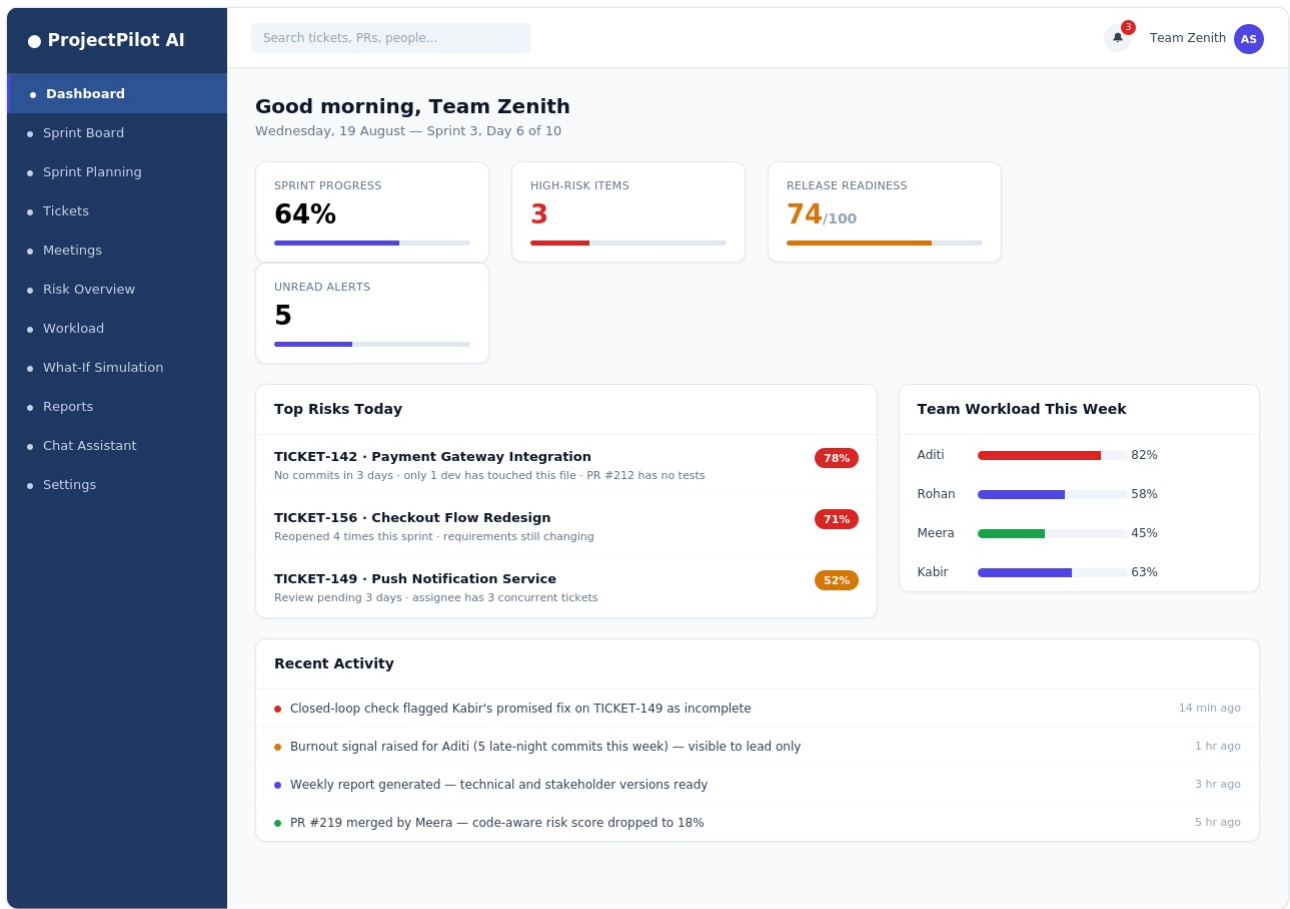
ANTIGRAVITY BUILD PROMPT

Build a Next.js App Router login page at `app/(auth)/login/page.tsx` using TypeScript and Tailwind CSS. Centered white card (max-width 400px) on a background that gradients from `#1F3864` at the top to `#F8FAFC`. Card contains: the text "ProjectPilot AI" in bold navy (`#1F3864`, 22px), a one-line subtitle in gray (`#64748B`, 13px) reading "Sign in to your team's dashboard", and a full-width button with solid `#4F46E5` background, white text, rounded-lg, that calls `supabase.auth.signInWithOAuth({ provider: 'github' })` on click using the Supabase client from `lib/supabase/client.ts`. Use the Inter font. No navbar or sidebar on this page.

5.2 Dashboard — `/dashboard`

Purpose: the one screen everyone checks first thing — sprint health, top risks, workload, recent activity, all at a glance.

Layout: greeting header → row of 4 stat cards (Sprint Progress, High-Risk Items, Release Readiness, Unread Alerts) → two-column row (Top Risks list left, Team Workload bars right) → full-width Recent Activity feed.



Data

React Query hooks: `useDashboardStats`, `useTopRisks`, `useWorkloadSummary`, `useRecentActivity`

Components

`StatCard`, `TopRisksList`, `WorkloadBarList`, `ActivityFeed` (all in `components/features/dashboard/`)

ANTIGRAVITY BUILD PROMPT

Build the Dashboard page at `app/(dashboard)/dashboard/page.tsx` (Next.js App Router, TypeScript, Tailwind). Layout: a greeting heading ("Good morning, {teamName}") with a gray subtitle showing today's date and sprint day count. Below it, a row of 4 equal-width stat cards (shadcn/ui Card component) showing: Sprint Progress (% with a thin progress bar in #4F46E5), High-Risk Items (count in #DC2626), Release Readiness (score/100 in #D97706), Unread Alerts (count). Below that, a two-column row: left column (62% width) is a card titled "Top Risks Today" listing tickets with a colored risk pill (red $\geq 70\%$, amber 40-69%, green $< 40\%$ — use the shared function in `lib/riskColor.ts`) and a one-line gray reason underneath each; right column (36% width) is a card titled "Team Workload This Week" with horizontal bar rows per team member. Below both, a full-width "Recent Activity" card listing timestamped events with a small colored status dot. Fetch all data via React Query hooks in `hooks/useDashboard.ts` pointed at the FastAPI backend. Use the navy/indigo/red/amber/green palette exactly as defined in `lib/riskColor.ts` and `tailwind.config.ts`.

5.3 Sprint Board (Kanban) — `/sprints/board`

Purpose: the day-to-day working view — drag tickets across To Do / In Progress / In Review / Done.

Layout: sprint selector + "Plan New Sprint" button at top; 4 columns below, each a scrollable list of ticket cards (title, assignee avatar, story points, small risk-pill if risk $\geq 40\%$).

Data	<code>useTickets(sprintId)</code> , mutation <code>useUpdateTicketStatus</code>
Components	<code>KanbanColumn</code> , <code>TicketCard</code> (shared with Ticket Detail's header)
State	Zustand <code>useFilterStore</code> for "assignee" quick-filter; drag state is local <code>useState</code> , not global

ANTIGRAVITY BUILD PROMPT

Build a Kanban sprint board at `app/(dashboard)/sprints/board/page.tsx`. Top bar: a sprint dropdown selector (shadcn/ui Select) and a "Plan New Sprint" button (solid #4F46E5) aligned right. Below: 4 equal-width columns titled "To Do", "In Progress", "In Review", "Done" with a light gray (#F1F5F9) header per column. Each column is a vertically scrollable list of ticket cards: white background, rounded-lg, 1px #E2E8F0 border, showing the ticket ID (gray, 10px), title (bold, 12px), a small circular assignee avatar with initials, story points as a small badge, and — only if risk score $\geq 40\%$ — a small colored risk pill top-right using `lib/riskColor.ts`. Implement drag-and-drop between columns using `@dnd-kit/core`, calling a `useUpdateTicketStatus` React Query mutation on drop. Fetch tickets via `useTickets(sprintId)` filtered by status.

5.4 Sprint Planning — `/sprints/plan`

Purpose: where the AI Task Generator and AI Sprint Planner actually run.

Layout: left: a large textarea for pasting requirements + "Generate Tasks" button; once generated, a review list of editable draft tickets with checkboxes; right/bottom: "Plan Sprint" button that arranges accepted tickets, showing the resulting balanced schedule per developer.

Data	Mutations <code>useGenerateTasks</code> , <code>usePlanSprint</code> (both call FastAPI endpoints that trigger the LangGraph agents)
Form	react-hook-form for the requirements textarea + per-ticket edit fields

ANTIGRAVITY BUILD PROMPT

Build the Sprint Planning page at `app/(dashboard)/sprints/plan/page.tsx`. Top: a card with a large textarea (placeholder "Paste your requirements here...") and a button "Generate Tasks with AI" (solid #4F46E5, shows a loading spinner while the `useGenerateTasks` mutation is pending). Once tasks return, render them as a list of editable cards below — each with an editable title (inline text input), editable description (textarea), a story-points number input, and a checkbox to include/exclude it from the sprint, using react-hook-form with a zod schema. Below the list, a "Plan Sprint" button that calls `usePlanSprint` with the accepted tickets, then renders a simple per-developer schedule summary (name, assigned story points, days) in a table once the response returns.

5.5 Ticket Detail — `/tickets/[id]`

Purpose: the single most important page in the app — this is where Explainable Risk Scores and Code-Aware Risk Detection actually get seen and trusted.

Layout: left column (66%): title/status header → large risk score panel with ranked, bar-weighted reasons → tabs (Overview / Linked Commits & PRs / Activity) → description → linked PRs list. Right column (32%): assignee, sprint, story points, priority, and a bus-factor-style note if relevant.

● ProjectPilot AI

● Dashboard

● Sprint Board

● Sprint Planning

● Tickets

● Meetings

● Risk Overview

● Workload

● What-If Simulation

● Reports

● Chat Assistant

● Settings

Sprint Board > TICKET-142

TICKET-142

Payment Gateway Integration

In Progress

78%

High Risk — Code-Aware

Last computed 14 minutes ago

WHY IS THIS RISKY? (EXPLAINABLE, NOT A BLACK BOX)

No commits on this ticket in 3 days +29%

Only 1 developer has ever touched payment_service.py +24%

PR #212 adds 640 lines with zero new tests +18%

Sprint velocity down 15% vs. last 3 sprints +7%

Overview

Linked Commits & PRs

Activity

Description

Integrate the Stripe payment gateway for checkout, including webhook handling for payment confirmation and refund flows. Must support INR and USD. Acceptance criteria: unit tests for success/failure/timeout paths, and a sandbox demo before merge.

Linked Pull Requests

#212 · Add Stripe payment service

+640/-12 lines 0 new tests Review pending 3d

ASSIGNEE

Aditi Sharma

SPRINT

Sprint 3 (Aug 14-24)

STORY POINTS

8

PRIORITY

High

BUS FACTOR NOTE

Only Aditi understands this module

CREATED

Aug 14, 2026

Data `useTicket(id)` , `useRiskScore(ticketId)` (returns score + `shap_explanation` JSON), `useLinkedPRs(ticketId)`

Components `RiskExplainPanel` is the star component — build it once, reuse on Risk Overview's row-expand too

ANTIGRAVITY BUILD PROMPT

Build the Ticket Detail page at `app/(dashboard)/tickets/[id]/page.tsx`, two-column layout (66% / 32%, both `shadcn/ui` Cards). Left column top: ticket ID (gray, 10px uppercase), title (bold, 19px), a status pill. Below it, a "Risk Explain Panel" component (`components/features/tickets/RiskExplainPanel.tsx`): a large risk percentage (34px, colored red/amber/green by `lib/riskColor.ts` thresholds) next to "High Risk — {risk_type}" text, then a "Why is this risky?" section listing each reason from the `shap_explanation` JSON as a row with the reason text, its % contribution, and a horizontal bar sized proportionally to that contribution (bar color matches severity). Below that, `shadcn/ui` Tabs for "Overview" / "Linked Commits & PRs" / "Activity", with Overview showing the ticket description and a list of linked PRs (each showing +lines/-lines, test coverage delta badge, and review status). Right column: labeled fields for Assignee, Sprint, Story Points, Priority, Created date, each as a small stacked label/value pair. Fetch data with `useTicket`, `useRiskScore`, and `useLinkedPRs` hooks.

5.6 Meetings — `/meetings` and `/meetings/[id]`

Purpose: upload/browse meeting recordings; view a transcript and its extracted, closed-loop-verified action items.

List layout: upload button top-right; list of meeting cards (title, date, duration, action-item count with a small "1 unverified" flag if relevant).

Detail layout: transcript on the left (scrollable, monospace-lite), extracted action items on the right as a checklist, each showing owner, due date, and a status badge (pending / verified done / flagged incomplete).

Data `useMeetings` , `useMeeting(id)` , `useActionItems(meetingId)` , mutation `useUploadMeeting`

ANTIGRAVITY BUILD PROMPT

Build two pages. (1) `app/(dashboard)/meetings/page.tsx`: a button "Upload Recording" top-right (opens a `shadcn/ui` Dialog with a file input calling `useUploadMeeting`), and below it a list of meeting cards showing title, date, and an action-item summary badge — amber "1 unverified" pill if any action item has status `flagged_incomplete`. (2) `app/(dashboard)/meetings/[id]/page.tsx`: two-column layout — left (60%) a card showing the transcript text in a scrollable area with JetBrains Mono font, 12px; right (38%) a card titled "Action Items" listing each item with owner avatar, description, due date, and a status badge colored gray (pending), green (verified_done), or red (flagged_incomplete). Fetch with `useMeeting(id)` and `useActionItems(id)`.

5.7 Risk Overview — /risk

Purpose: every open risk across the whole team, in one sortable table — the "control room" view.

Layout: filter chips (All / Code-Aware / Delay / current sprint) → full-width table: Item, Type tag, Risk Score pill, Top Reason, Assignee, Updated. Clicking a row expands the same RiskExplainPanel used on Ticket Detail, inline.

● ProjectPilot AI

● Dashboard

● Sprint Board

● Sprint Planning

● Tickets

● Meetings

● Risk Overview

● Workload

● What-If Simulation

● Reports

● Chat Assistant

● Settings

Risk Overview

Risk Overview

Every open risk across the team, ranked by severity — explainable, not a black box.

All Types

Code-Aware

Delay

Sprint 3 only

ITEM	TYPE	RISK SCORE	TOP REASON	ASSIGNEE	UPDATED
TICKET-142 · Payment Gateway Integration	CODE-AWARE	78%	No commits in 3 days, only 1 dev knows this file	AS Aditi	14 min ago
TICKET-156 · Checkout Flow Redesign	DELAY	71%	Reopened 4 times, requirements still changing	RV Rohan	32 min ago
PR #212 · Add Stripe payment service	CODE-AWARE	74%	640 lines changed, zero new tests added	AS Aditi	1 hr ago
TICKET-149 · Push Notification Service	DELAY	52%	Review pending 3 days, assignee overloaded	KB Kabir	3 hr ago
Meera Iyer · workload signal	BURNOUT	48%	Visible to team lead only — see Workload page	MI Meera	5 hr ago
TICKET-138 · User Profile Page	DELAY	22%	On track, normal commit pace	RV Rohan	6 hr ago

Data useAllRiskScores(filters)

State Zustand useFilterStore holds the active chip/filter

ANTIGRAVITY BUILD PROMPT

Build the Risk Overview page at app/(dashboard)/risk/page.tsx. Top: a row of filter chips ("All Types", "Code-Aware", "Delay", "Sprint 3 only") built with shadcn/ui Badge/Button-toggle styling, active chip filled navy (#1F3864), inactive chips white with gray border — clicking one updates useFilterStore (Zustand) and refetches useAllRiskScores(filters). Below: a full-width Card containing a table with columns Item, Type (small colored tag: indigo for code-aware, blue for delay, purple for burnout), Risk Score (colored pill per lib/riskColor.ts), Top Reason (gray text), Assignee (small avatar + name), Updated (relative time). Clicking a row expands it inline to show the same RiskExplainPanel component used on the Ticket Detail page, reused as-is, not rebuilt.

5.8 Workload — /workload

Purpose: team capacity view, plus the lead-only burnout panel.

Layout: a bar chart of workload % per person (visible to everyone) → below it, a separate card titled "Wellbeing Signals" visible **only if the logged-in user's** role_in_team = 'lead', listing private burnout flags with the same explain-panel pattern, in purple instead of red/amber/green.

Data useWorkload(teamId) (everyone), useBurnoutSignals(teamId) (only called/rendered if lead — RLS also blocks it server-side per the database guide)

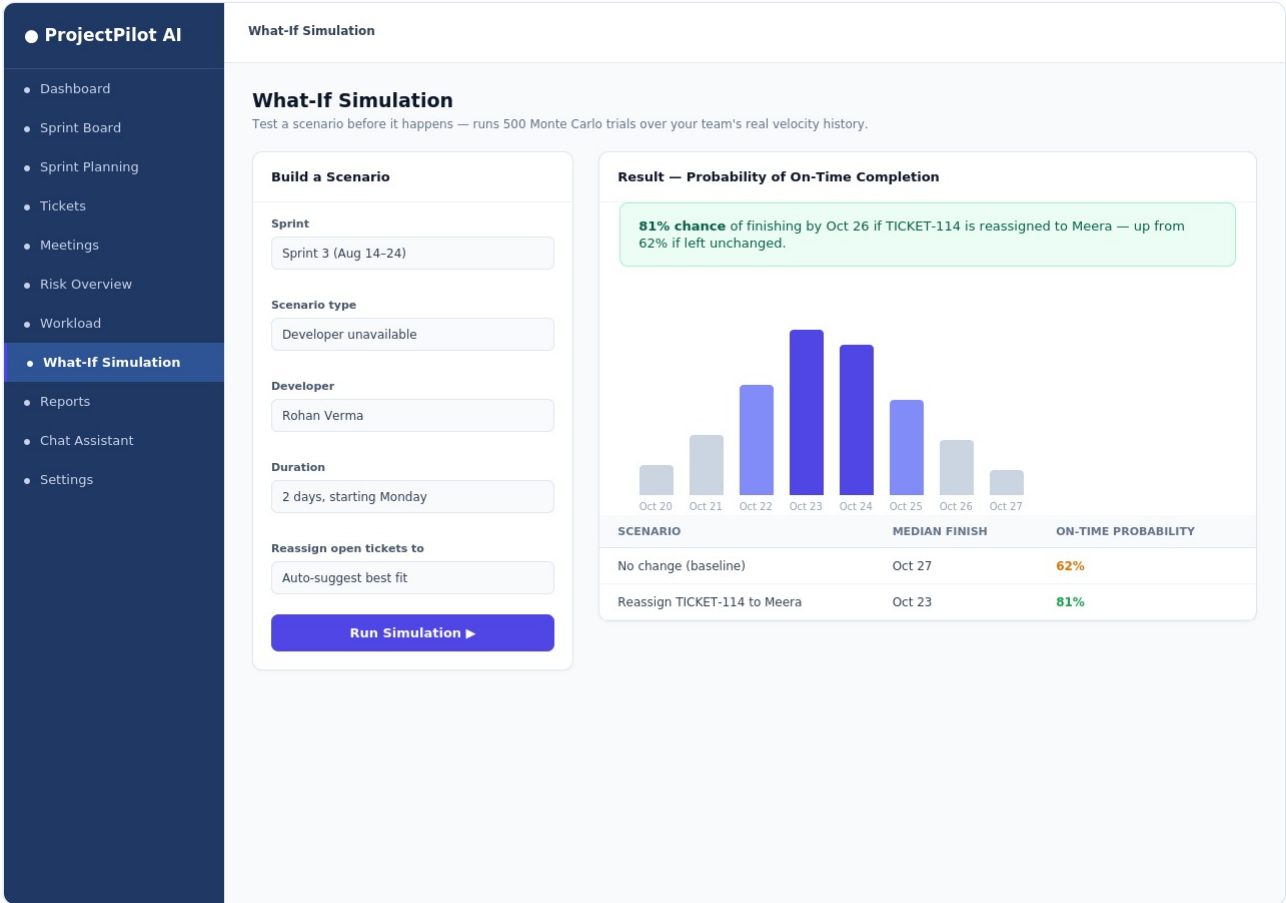
ANTIGRAVITY BUILD PROMPT

Build the Workload page at `app/(dashboard)/workload/page.tsx`. Top: a Recharts horizontal BarChart showing each team member's workload % for the current week, bars colored #4F46E5 (or #DC2626 if over 80%). Below it, conditionally render a card titled "Wellbeing Signals" ONLY when the current user's team role is 'lead' (check via a `useCurrentUserRole` hook) — inside, list any burnout risk_scores entries with a purple (#A21CAF) accent instead of the usual red/amber/green, each showing the person, the score, and its shap_explanation reasons in the same bar-weighted style as RiskExplainPanel. If the user is not a lead, this card must not render at all, not just be hidden with CSS.

5.9 What-If Simulation — `/simulation`

Purpose: the Monte Carlo "what if" tool.

Layout: left (32%): scenario builder form (sprint, scenario type, developer, duration, reassignment). Right (65%): a plain-English result banner, a histogram of possible completion dates, and a comparison table (baseline vs. scenario).



Data Mutation `useRunSimulation` ; result rendered from its response, not persisted state

Chart Recharts `BarChart` for the probability histogram

ANTIGRAVITY BUILD PROMPT

Build the What-If Simulation page at `app/(dashboard)/simulation/page.tsx`, two-column layout (32% / 65%). Left: a Card titled "Build a Scenario" with a react-hook-form containing: Sprint select, Scenario Type select ("Developer unavailable", "Ticket delayed", etc.), Developer select, Duration input, and a "Reassign open tickets to" select — below the form, a full-width button "Run Simulation" (solid #4F46E5) that triggers the `useRunSimulation` mutation. Right: a Card titled "Result — Probability of On-Time Completion" containing: a green-tinted (#ECFDF5 background, #065F46 text) result banner sentence generated from the response, a Recharts BarChart histogram of completion-date probabilities (bars colored #818CF8 to #4F46E5 by height/likelihood), and a comparison table below with columns Scenario / Median Finish / On-Time Probability, coloring the probability green if $\geq 75\%$, amber 50-74%, red $< 50\%$.

5.10 Reports — `/reports`

Purpose: Audience-Aware Auto Reports — same data, two written versions.

Layout: a week selector, a toggle ("Technical" / "Stakeholder"), and the selected report body rendered as clean formatted text below.

Data

`useReport(weekStart, version)`

ANTIGRAVITY BUILD PROMPT

Build the Reports page at `app/(dashboard)/reports/page.tsx`. Top bar: a week selector dropdown and a shadcn/ui Tabs toggle with two options "Technical" and "Stakeholder". Below, a Card rendering the selected report's text (`technical_version_text` or `stakeholder_version_text` from `useReport`) as formatted prose — use a slightly larger line-height (1.7) since this is meant to be read, not scanned. Include a "Copy to clipboard" and "Download as PDF" button top-right of the card.

5.11 Chat Assistant — `/chat`

Purpose: ask the RAG chatbot direct questions, grounded in real project data.

Layout: standard chat UI — scrollable message list (user right-aligned, assistant left-aligned with a small source-reference chip under any answer that cites specific tickets/reports), input box pinned to bottom.

Data

`useChatHistory`, mutation `useSendMessage`

State

Zustand holds the in-progress draft text before send

ANTIGRAVITY BUILD PROMPT

Build the Chat Assistant page at `app/(dashboard)/chat/page.tsx`. Full-height layout: a scrollable message list area, user messages right-aligned in a solid #4F46E5 bubble with white text, assistant messages left-aligned in a white bubble with #E2E8F0 border. Under any assistant message that references specific tickets/reports, render small gray "source chips" (e.g. "TICKET-142", "Weekly Report Aug 14") that link to those pages. Bottom: a fixed input bar with a text input (bound to a Zustand draft store) and a send button (#4F46E5), calling `useSendMessage` on submit and appending the response to `useChatHistory`'s cache optimistically.

5.12 Settings / Integrations — `/settings/integrations`

Purpose: connect GitHub/Jira, manage team members.

Layout: a card per integration (GitHub, Jira) showing connection status and a Connect/Disconnect button; below, a simple team member list with role badges (member/lead).

ANTIGRAVITY BUILD PROMPT

Build the Settings page at `app/(dashboard)/settings/integrations/page.tsx`. Top: two Cards side by side, one for GitHub and one for Jira, each showing a status pill ("Connected" green or "Not Connected" gray), the connected repo/project name if applicable, and a Connect or Disconnect button. Below: a full-width Card titled "Team Members" listing each member's avatar, name, email, and a role badge ("Lead" in navy, "Member" in gray) — only visible/editable by users whose own role is 'lead'.

6. Global Rules for the Whole Team

- **Never hardcode a risk color.** Always import from `lib/riskColor.ts`. If a threshold ever needs to change, it should change in exactly one file.
- **Never fetch data inside a component that also renders UI logic.** Data fetching lives in `hooks/`, components receive data as props.
- **Never put server data in a Zustand store.** If it comes from the database, it belongs in React Query.
- **Reuse `RiskExplainPanel` everywhere risk is shown** (Ticket Detail, Risk Overview, Workload's burnout panel) — one component, three places, so the explanation always looks and behaves identically.
- **Every new page starts from the same shell layout** (`app/(dashboard)/layout.tsx`) — nobody rebuilds the sidebar/topbar per page.

This document is the complete, final frontend reference for ProjectPilot AI — architecture, design system, and every page spec with a ready-to-use build prompt. Refer back to it for every new page or component the team builds.